

Labo 02 — Labo pratique : disséquer une image, monter un registry

Objectif : manipuler tags, couches et digests, puis publier une image dans un registry privé que vous ferez tourner vous-même.

Prérequis — Labo 01 terminé. Le port `5000` doit être libre sur votre machine (`ss -lntp | grep :5000` ne doit rien renvoyer).

Fichiers fournis — `files/Dockerfile` (deux lignes, expliquées au labo 04 ; ici on s'en sert seulement comme générateur d'images).

Étape 1 — Lire un nom d'image

```
docker pull nginx:alpine
docker pull alpine
```

Observez les lignes `Pull complete` et le `Digest: sha256:...` en fin de sortie.

```
docker image inspect --format '{{.RepoTags}}'          nginx:alpine
docker image inspect --format '{{index .RepoDigests 0}}' nginx:alpine
```

Observez d'un côté `[nginx:alpine]`, de l'autre `nginx@sha256:4a73073bd557c65b7595...`.

Explication. `nginx:alpine` est un nom lisible et déplaçable. Le digest est l'identité réelle et permanente du contenu. Notez ce digest quelque part : il servira à l'étape 7.

Étape 2 — Deux façons de lister ses images

```
docker images
```

Observez les colonnes `IMAGE`, `ID`, `DISK USAGE`, `CONTENT SIZE`. C'est la présentation du magasin d'images **containerd**, utilisé par les versions récentes de Docker.

```
docker info | grep -iE 'storage driver|driver-type'
```

Observez `Storage Driver: overlayfs` et `driver-type: io.containerd.snapshotter.v1`.

La plupart des tutoriels montrent l'ancien affichage. Vous pouvez le reproduire :

```
docker images --format 'table {{.Repository}}\t{{.Tag}}\t{{.ID}}\t{{.Size}}'
```

Observez les colonnes classiques `REPOSITORY / TAG / IMAGE ID / SIZE`.

Explication. Prenez l'habitude de `--format` : il rend vos commandes indépendantes des changements de présentation, et exploitables en script.

Étape 3 — Les couches, et où passe le poids

```
docker history nginx:alpine --format 'table {{.Size}}\t{{.CreatedBy}}'
```

Observez une seule grosse couche (51.8MB , l'installation d'nginx), quelques petites couches COPY de scripts, et de nombreuses lignes à 0B .

Explication. Les lignes à 0B sont des instructions de **métadonnées** : ENV , CMD , EXPOSE , ENTRYPOINT , STOPSIGNAL . Elles ne créent aucun fichier. Cette commande est votre premier réflexe quand une image est anormalement lourde : la ligne coupable saute aux yeux.

Comparez le coût réel de vos images :

```
docker system df
docker system df -v | head -n 15
```

Observez dans docker system df que SIZE (occupation réelle) est très inférieur à la somme des tailles individuelles, et repérez la colonne SHARED SIZE du mode verbeux.

Étape 4 — docker tag ne copie rien

```
docker tag nginx:alpine mon-nginx:v1
docker tag nginx:alpine mon-nginx:preprod
docker images --format 'table {{.Repository}}\t{{.Tag}}\t{{.ID}}' | grep -E 'nginx'
```

Observez trois lignes... avec le même IMAGE ID .

```
docker rmi mon-nginx:v1
```

Observez la sortie : uniquement Untagged: mon-nginx:v1 . Aucun Deleted: .

```
docker rmi mon-nginx:preprod
```

Observez cette fois encore Untagged: seulement — car nginx:alpine désigne toujours l'image.

Explication. Un tag est une référence. Tant qu'il reste un nom, les données restent. C'est pourquoi « j'ai fait des docker rmi et je n'ai pas récupéré de place » est une plainte fréquente et parfaitement normale.

Étape 5 — Construire deux versions d'une même image

Copiez le fichier fourni dans un dossier de travail :

```
mkdir -p ~/labo-docker/02 && cd ~/labo-docker/02
cp <chemin-du-labo>/files/Dockerfile .
cat Dockerfile
```

```
docker build -t demo-couches:1.0 .
docker history demo-couches:1.0 --format 'table {{.ID}}\t{{.Size}}\t{{.CreatedBy}}'
```

Observez trois lignes : votre `RUN` (quelques kilo-octets), le `CMD` de l'image de base (`0B`), et l'ajout du système de fichiers Alpine (`~9MB`).

Modifiez la version puis reconstruisez sur **le même tag** :

```
sed -i 's/version 1/version 2/' Dockerfile
docker build -t demo-couches:1.0 .
docker run --rm demo-couches:1.0 cat /version.txt
```

Observez `version 2`, et un nouvel `IMAGE ID` pour le même tag.

Explication. Le tag `demo-couches:1.0` a été **déplacé** vers une nouvelle image : exactement le scénario de la question 2. Rien n'avertit l'utilisateur.

```
docker images --filter dangling=true
```

Observez que la liste est probablement **vide**. Avec le magasin d'images containerd, l'ancienne image est déréférencée puis nettoyée automatiquement. Sur un Docker plus ancien (ou avec le magasin historique), vous verriez ici une image `<none>:<none>` occupant toujours du disque : c'est ce qu'on appelle une image *dangling*.

Étape 6 — Monter un registry privé

Un registry n'est rien d'autre qu'un conteneur :

```
docker run -d -p 5000:5000 --name registry-labo registry:2
docker ps --filter name=registry-labo
curl -s http://localhost:5000/v2/_catalog
```

Observez `{"repositories":[]}` : le registry est vide et fonctionnel.

Publiez-y votre image :

```
docker tag demo-couches:1.0 localhost:5000/socle/demo:1.0
docker push localhost:5000/socle/demo:1.0
```

Observez la progression couche par couche, puis un `Info -> Not all multiplatform- content is present...` : normal, vous ne poussez que l'architecture de votre machine.

```
curl -s http://localhost:5000/v2/_catalog
curl -s http://localhost:5000/v2/socle/demo/tags/list
```

Observez `{"repositories":["socle/demo"]}` puis `{"name":"socle/demo","tags":["1.0"]}`.

Explication. Vous venez de reproduire, en trois commandes, ce que fait la CI de votre entreprise. Un registry est un service HTTP avec une API standardisée (`/v2/...`) : Harbor, ECR ou GitLab Registry parlent exactement le même protocole.

Vérifiez le transfert différentiel :

```
docker tag demo-couches:1.0 localhost:5000/socle/demo:1.1
docker push localhost:5000/socle/demo:1.1
```

Observez des `Layer already exists` : rien n'est retransféré, seul le manifeste est écrit.

Étape 7 — Tirer par digest

Récupérez le digest tel que le registry le connaît :

```
curl -sI -H "Accept: application/vnd.oci.image.manifest.v1+json" \  
http://localhost:5000/v2/socle/demo/manifests/1.0 | grep -i docker-content-digest
```

Observez une ligne `docker-content-digest: sha256:...`. Copiez cette valeur.

```
docker rmi localhost:5000/socle/demo:1.0  
docker pull localhost:5000/socle/demo@sha256:<collez_ici>  
docker images --format 'table {{.Repository}}\t{{.Tag}}\t{{.ID}}' | grep demo
```

Observez que l'image est tirée et référencée par son digest, avec un tag `<none>`.

Explication. C'est la forme utilisée par les déploiements sérieux : elle est **infalsifiable**. Même si quelqu'un republie `socle/demo:1.0` avec un autre contenu, votre digest continue de désigner l'image que vous avez testée.

Étape 8 — Transporter une image sans réseau

```
docker save -o /tmp/demo.tar demo-couches:1.0  
ls -lh /tmp/demo.tar
```

Observez une archive de quelques Mo.

```
tar -tf /tmp/demo.tar | head -n 8
```

Observez des fichiers `blobs/sha256/...`, `index.json`, `manifest.json` : ce sont bien les couches et les métadonnées.

Comparez avec `export`, qui travaille sur un **conteneur** :

```
docker run -d --name tmpx nginx:alpine  
docker export tmpx | docker import - nginx-aplati:v1  
docker image inspect --format '{{json .Config.Cmd}}' nginx-aplati:v1
```

Observez `null` : l'image importée **ne sait plus quoi lancer**.

```
docker run --rm nginx-aplati:v1
```

Observez l'erreur `no command specified`.

Explication. Retenez la règle : `save` / `load` pour une image, `export` / `import` jamais pour du déploiement.

Nettoyage

```
docker rm -f tmpx registry-labo
docker rmi nginx-aplati:v1 demo-couches:1.0 localhost:5000/socle/demo:1.1 registry:2
docker images --format '{{.Repository}}:{{.Tag}}' | grep -E 'demo|aplati|registry'
rm -f /tmp/demo.tar
```

Il peut rester l'image tirée par digest à l'étape 7 :

```
docker images --format 'table {{.ID}}\t{{.Repository}}' | grep localhost:5000
docker rmi <ID>
```

Observez qu'il ne reste que `alpine` et `nginx:alpine`, conservés pour la suite.

Ce que vous devez pouvoir affirmer maintenant

- Un tag est une référence déplaçable ; le digest identifie un contenu.
- `docker history` révèle où passe le poids d'une image.
- `docker tag` ne duplique rien ; `docker rmi` retire d'abord un nom.
- Un `push` ne transfère que les couches absentes du registry.
- Un registry est un simple service HTTP, montable en une commande.
- `export` / `import` détruit la configuration de l'image ; `save` / `load` la préserve.